



Inženir Računalništva in Informatike

VAJA

Kranj, junij 2026

Miha Gradišar



Inženir Računalništva in Informatike

Vaja višjega strokovnega izobraževanja

Algoritem A*

Avtor: Miha Gradišar

Kranj, junij 2026

Kazalo vsebine

1	UVOD	1
2	OPIS PROBLEMA.....	2
3	METODOLOGIJA.....	3
3.1	Algoritem A*	3
3.2	Strukture podatkov.....	3
3.3	Ključne funkcije.....	3
4	REZULTATI.....	4
4.1	Analiza optimalnosti.....	5
4.2	Časovna in prostorska zahtevnost.....	5
5	ZAKLJUČEK	6

Kazalo slik

Slika 1: Cestni graf z razdaljami in označeno optimalno potjo	2
Slika 2: Vrednosti g , h in f po korakih	5

1 UVOD

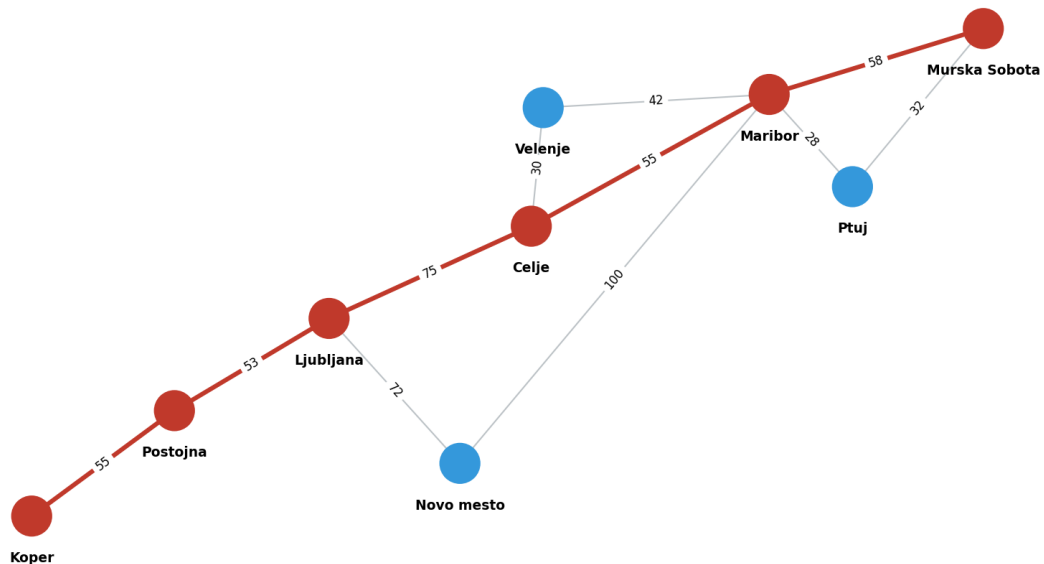
A* je algoritem za iskanje najcenejše poti v grafu. Od navadnega slepega iskanja se loči po tem, da si pri vsakem koraku pomaga s hevristiko, torej z grobo oceno, kako daleč je še do cilja. Zaradi te ocene ne tava na vse strani, ampak gre bolj naravnost proti cilju. Ravno to me je pri vaji najbolj zanimalo, kako močno taka ocena skrči iskanje. Za primer sem vzel majhno cestno omrežje med nekaj slovenskimi mesti in iskal najkrajšo pot od Kopra do Murske Sobote.

Omrežje sem pustil majhno nalašč, da lahko v poročilu prikažem vsak korak posebej in se vidi, kako algoritem v resnici dela. A* sicer ni vezan na zemljevide, enako se uporablja v igrah za iskanje poti po mreži in pri raznih ugankah, le graf in hevristika sta drugačna.

2 OPIS PROBLEMA

Vozlišča v grafu so mesta, povezave pa ceste med njimi. Na vsaki povezavi piše cestna razdalja v kilometrih. Številke so približne, za vajo to ni pomembno. Start je Koper, cilj Murska Sobota. Poleg cen na povezavah A* potrebuje še heuristiko $h(n)$. To je groba ocena, kako daleč je posamezno mesto še od cilja. Zanj sem vzel kar oceno zračne razdalje do Murske Sobote. Zračna razdalja je vedno krajša od cestne, zato heuristika nikoli ne preceni prave cene. To je edini pogoj, ki ga A* res potrebuje, da na koncu vrne optimalno pot.

Cestni graf z označeno optimalno potjo (rdeca)



Slika 1: Cestni graf z razdaljami in označeno optimalno potjo

Vozlišče	$h(n)$ km
Koper	250
Postojna	210
Ljubljana	160
Celje	95
Novo mesto	135
Velenje	85
Maribor	50
Ptuj	28
Murska Sobota	0

3 METODOLOGIJA

3.1 ALGORITEM A*

A* za vsako mesto vodi tri številke. $g(n)$ je cena že prehojene poti od starta do tega mesta. $h(n)$ je ocena, koliko je do cilja še ostalo. $f(n)$ je njuna vsota in pove, kako blizu cilju je iti naprej čez to mesto. V vsakem koraku algoritem vzame iz seznama OPEN mesto z najmanjšim f in ga obdela. Ker f združuje že prehojeno pot in oceno preostanka, se iskanje samo od sebe nagiba proti cilju.

Seznam OPEN so mesta, ki še čakajo na obdelavo, CLOSED pa mesta, ki so že obdelana. Ko algoritem vzame mesto iz OPEN, ga premakne v CLOSED in pogleda njegove sosedo. Za vsakega izračuna novo g . Če je ta manjša od prej znane ali soseda še ni videl, si zapomni novo ceno in od kod je do njega prišel, soseda pa doda v OPEN. To ponavlja, dokler iz OPEN ne potegne cilja.

3.2 STRUKTURE PODATKOV

Graf je navaden slovar, kjer vsako mesto kaže na seznam sosedov in razdalj do njih. Hevristike so v drugem slovarju. OPEN sem naredil s `heapq`, ker mi ta z eno operacijo vrne mesto z najmanjšim f in mi ni treba ničesar sortirati ročno. Za rekonstrukcijo poti je tu še slovar `stars`, ki za vsako mesto hrani, iz katerega mesta je algoritem prišel do njega.

3.3 KLJUČNE FUNKCIJE

Glavna funkcija je `a_zvezda`, ki naredi iskanje in vrne pot, njeno ceno in vrstni red obdelave. Funkcija `rekonstruiraj` na koncu po slovarju `stars` sestavi pot nazaj od cilja do začetka. Ostalo sta dve funkciji za risanje grafov.

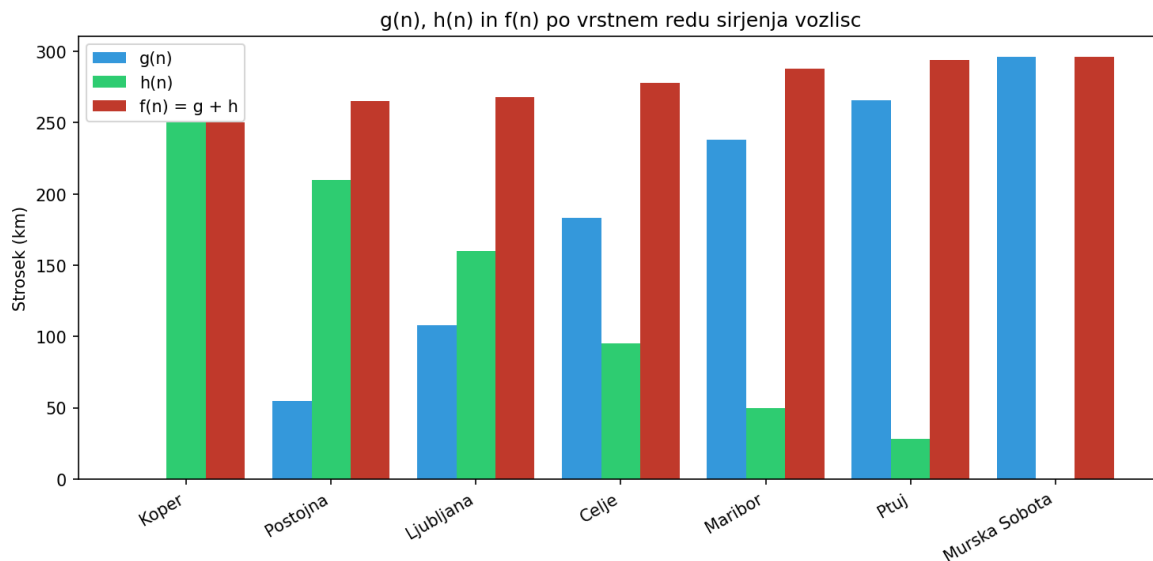
4 REZULTATI

A* je do cilja prišel v sedmih korakih. Spodnja tabela kaže, kaj se je dogajalo. V vsaki vrstici je mesto, ki ga je algoritem ravno obdelal, njegove g, h in f, ter kaj je po tem ostalo v OPEN.

Korak	Razširjeno	g	h	f	OPEN po koraku (urejeno po f)
1	Koper	0	250	250	Postojna (265)
2	Postojna	55	210	265	Ljubljana (268)
3	Ljubljana	108	160	268	Celje (278), Novo mesto (315)
4	Celje	183	95	278	Maribor (288), Velenje (298), Novo mesto (315)
5	Maribor	238	50	288	Ptuj (294), Murska Sobota (296), Velenje (298), Novo mesto (315)
6	Ptuj	266	28	294	Murska Sobota (296), Velenje (298), Novo mesto (315)
7	Murska Sobota	296	0	296	cilj dosežen

CLOSED se polni po istem vrstnem redu, kot algoritem obdeluje mesta, torej Koper, Postojna, Ljubljana, Celje, Maribor, Ptuj in Murska Sobota.

Ena stvar mi je padla v oči. Algoritem ni šel naravnost na cilj, ampak je vmes obdelal še Ptuj. Če pogledam tabelo, je razlog jasen. Ptuj je imel f enak 294, Murska Sobota pa 296, zato je Ptuj prišel na vrsto prej. A* pač ne skoči na cilj takoj, ko ga zagleda, ampak prej pregleda vse, kar je videti vsaj tako blizu kot cilj. Velenja in Novega mesta sploh ni odprl, ker sta imela previsok f. In točno tu se vidi, čemu heuristika sploh služi, brez nje bi iskanje zašlo tudi v ti dve smeri.



Slika 2: Vrednosti g, h in f po korakih

Najcenejša pot gre od Kopra čez Postojno, Ljubljano, Celje in Maribor do Murske Sobote in skupaj nanese 296 km.

4.1 ANALIZA OPTIMALNOSTI

Hevristika nikoli ne preceni prave razdalje, kar pri A* zadošča za garancijo, da je najdena pot optimalna. Za vsak primer sem dve drugi poti preračunal še sam. Pot čez Velenje nanese 315 km, pot čez Novo mesto pa 340 km. Obe sta dražji od poti čez Maribor, tako da se ujema.

4.2 ČASOVNA IN PROSTORSKA ZAHTEVNOST

V najslabšem primeru je A* eksponenten, $O(b^d)$ na potenco d), ampak to je teoretična meja, do katere v praksi z dobro hevristikto skoraj nikoli ne pride. Pri meni je od devetih mest obdelal samo sedem. Bolj kot čas me skrbi pomnilnik, ker mora algoritem ves čas hraniti oba seznama, OPEN in CLOSED, ta dva pa se pri velikih grafih hitro napolnita.

5 ZAKLJUČEK

A* je na mojem grafu brez težav našel optimalno pot in pri tem obdelal le sedem mest. Koliko dela si prihrani, je odvisno skoraj samo od hevristike. Če ta preveč podcenjuje, se algoritem vse bolj obnaša kot navadno slepo iskanje, če bi precenjevala, pa bi lahko zgrešil najboljšo pot. Ocena zračne razdalje je tu nekje vmes, dovolj dobra, da usmerja, in dovolj previdna, da ostane varna. Da se izboljšati. Če bi hevristiko računal iz pravih koordinat mest namesto na oko, bi bila natančnejša in A* bi verjetno obdelal še manj mest.